SRS: Hover, a 3D Drone Flight Simulator

Team 1

Team Members: Taha Oguzhan Ucar, Muhammed Bilal, Vamsi Mannava, Sophakotra Son

Problem Statement

People who want to understand how drones fly, or test how a drone would handle a mission, usually face two bad options. Real drones are expensive, risky to crash, and limited by weather and flight regulations. Existing simulators are either full game engines that are too complex to learn and extend, or toy programs that ignore real physics like gravity, drag, wind, and battery limits. What we need is a simulator that is simple to use but physically honest: where a user can set up drones, give them missions, change the wind, and watch them climb, hover, fly to waypoints, run low on battery, and land or crash, all based on real physics. It should also be built with a clean object-oriented design so new drone types and features can be added without rewriting the system.

Objective / Goal

We're building Hover, a desktop application in Java that simulates drones flying in a 3D world under real physics. Users can add different types of drones, plan missions made of steps like take off, fly to a point, hover, and land, and watch the results in 2D and 3D views. The simulation models gravity, thrust, air drag, wind with gusts, ground contact, collisions, payload changes, and battery drain. An event system reports important moments such as low battery, crashes, collisions, and completed missions, and the system can react to them automatically, for example by sending a drone home when its battery runs low. The design is built around clear interfaces and design patterns so that drones, forces, controllers, and views can each be extended independently, and the core model is fully covered by automated tests.

One Use Case (Primary)

User: A user running Hover on their desktop (for example, a student testing a delivery route)

Use Cases:

1. The user opens Hover and sees an empty 3D world with a ground plane.
2. The user adds a quadcopter at a starting position, checks its specifications (mass, maximum thrust, battery capacity), and attaches a 0.5 kg payload.
3. The user sets the wind to 5 m/s from the west with light gusts.
4. The user builds a mission in the mission editor: take off to 15 m, fly to two waypoints, drop the payload at the second waypoint, hover for 10 seconds, return, and land.

Hover checks that the quadcopter can perform every step, and the user saves the mission to a file for later.

5. The user starts the simulation. Each step, the drone's flight controller decides its thrust, and the physics engine moves the drone according to gravity, thrust, drag, and wind.
6. The user watches the flight in the 3D view and in the top-down and side views, while the status panel shows altitude, speed, throttle, battery level, mass, and the current mission step.
7. After the payload drop, the drone becomes lighter and needs less thrust to hover, which the status panel reflects immediately.
8. Fighting the wind drains the battery faster than planned. When the battery drops below 20%, Hover raises a low-battery event, and the drone automatically abandons the rest of its mission and returns home.
9. The drone lands safely. The event log records the payload drop, the low-battery warning, the return-home decision, and the landing. The user can reset and rerun the same scenario and get identical results.

Scope of the Project

In Scope:

- 3D simulation world with a flat ground plane, obstacles, and no-fly zones
- Custom physics engine with gravity, thrust, quadratic air drag, ground contact, and crash detection
- Wind with a constant base speed and random gusts, with gusts generated from a fixed seed so runs are repeatable
- Air density that decreases with altitude and affects drag
- Battery model where energy use depends on throttle
- Payloads that change a drone's mass when attached or released
- Multiple drone types: quadcopter (core), hexacopter and fixed-wing with lift and stall (extended)
- Flight controllers: hover, altitude hold, waypoint navigation, return to home, and fixed-wing flight
- Missions built from steps (take off, go to, hover, drop payload, land), created in a mission editor, saved to files, and loaded from files
- Mission validation against each drone's capabilities
- Drone lifecycle states (idle, taking off, flying, landing, landed, crashed) that block illegal actions
- Event system that reports take-off, landing, crashes, collisions, low battery, battery depleted, mission progress, and no-fly zone entry and exit
- Automatic reactions to events, such as returning home on low battery
- Collision detection between drones and between drones and obstacles, where any collision counts as a crash
- JavaFX user interface with 2D top-down and side views and a 3D view
- Simulation controls: start, pause, reset, and simulation speed
- Deterministic simulation: the same scenario always produces the same result

Optional and Stretch Goals (only if the core is complete):

- Higher-accuracy RK4 integration method
- Tilt-based flight model, where a drone tilts to move sideways
- ode4j physics adapter as an alternative physics engine
- jMonkeyEngine renderer as an alternative 3D view

Out of Scope:

- Physical collision response such as bouncing (collisions are treated as crashes)
- Full rotational rigid-body dynamics (no quaternions, inertia, or motor-level control)
- Turbulence models, ground effect, and terrain (the ground is a flat plane)
- Networking, multiplayer, or real drone hardware control

Tech Stack

Language and Platform:

- Language: Java
- JDK: JDK 25 (LTS), Temurin builds for macOS on Apple Silicon and Windows on x64

Build and IDE:

- Build Tool: Maven with the Maven Wrapper
- IDE: IntelliJ IDEA

User Interface:

- Framework: JavaFX 25 (controls, 2D canvas views, 3D view)
- Plugin: javafx-maven-plugin

Simulation:

- Physics Engine: custom engine written by the team, behind a PhysicsEngine interface
- Integration Method: semi-implicit Euler with a fixed time step of 0.01 seconds
- Math: custom immutable Vector3D class
- Optional Physics Adapter: ode4j

Architecture and Design:

- Architecture: Model-View-Controller (MVC)
- Design Patterns: Strategy, State, Command, Observer, Template Method, Builder, Decorator, Facade, Null Object
- Event System: Observer pattern with immutable event records and end-of-step delivery

Testing:

- Unit Testing: JUnit 5
- Code Coverage: JaCoCo (version 0.8.14 or newer)

Development Tools:

- Version Control: Git with GitHub (branch per task, protected main branch, pull requests with one review)
- Project Management: GitHub Issues and GitHub Projects
- CI/CD: GitHub Actions (builds and tests on both macOS and Windows)
- Diagrams: UML class and sequence diagrams written in Mermaid and stored in the repository

Hosting/Deployment:

- Desktop application, no server or cloud hosting required
- Distributed through the GitHub repository and run with the Maven Wrapper
- Optional: platform installers built with jpackage

External Interface Requirements

User Interfaces

- **Main Window:** Hover opens to the simulation view in the center, with control and status panels around it
- **Simulation View:** Switchable between a 3D view with an orbit camera and 2D top-down and side views
- **Control Panel:** Start, pause, reset, simulation speed, add or remove drones, wind settings
- **Drone Status Panel:** Live altitude, speed, throttle, battery level, mass, state, and current mission step for the selected drone
- **Mission Editor:** Build a mission step by step, with invalid steps flagged for the selected drone, and save or load mission files
- **Event Log:** Time-stamped list of events such as low battery, crash, collision, payload drop, and mission complete
- **Console Mode:** Text-only output of drone states, used for testing and debugging

Hardware Interfaces

- No specific hardware interfaces required
- Desktop or laptop running macOS on Apple Silicon or Windows on x64
- Graphics hardware supported by JavaFX (Metal or OpenGL on macOS, Direct3D on Windows)
- Keyboard and mouse for input

Software Interfaces

- **Java Runtime:** JDK 25
- **JavaFX Runtime:** Loaded through Maven for each operating system

- **File System:** Plain-text mission files for saving and loading missions
- **Physics Interface:** PhysicsEngine interface, so the custom engine can be replaced by an adapter (such as ode4j) without changing other code
- **Event Interface:** SimulationListener interface, through which the model delivers events to the views, the event log, and automatic reaction policies
- **View Interface:** Views receive read-only snapshots of the simulation and never modify the model directly

Functional Requirements

1. Create drones of different types with configurable mass, maximum thrust, drag, battery capacity, and starting position
2. Advance the simulation in fixed time steps of 0.01 seconds, independent of screen frame rate
3. Apply gravity, thrust, and air drag to every drone each step, and update its position and velocity
4. Reduce air density with altitude and apply it to drag calculations
5. Apply wind with a constant base speed and seeded random gusts, affecting drones through their speed relative to the air
6. Detect ground contact using vertical speed: land safely at or below 3 m/s downward speed, and crash above it
7. Drain battery energy based on throttle, and cut thrust when the battery is empty
8. Attach and release payloads, and update the drone's mass immediately
9. Control drones automatically with hover, altitude hold, waypoint, return-to-home, and fixed-wing flight controllers
10. Run missions as ordered steps (take off, go to, hover, drop payload, land), created in the app, saved to a file, or loaded from a file
11. Validate missions against each drone's capabilities, and reject steps a drone cannot perform, such as hovering for a fixed-wing drone
12. Enforce drone lifecycle states and reject illegal actions, such as taking off after a crash
13. Detect collisions between drones and with obstacles, and treat every collision as a crash
14. Detect when a drone enters or leaves a no-fly zone
15. Report events (take-off, landing, crash, collision, low battery, battery depleted, mission step done, mission complete, zone entered, zone exited) to all registered listeners, exactly once per occurrence, in a consistent order
16. React to events automatically through configurable policies, such as returning home on low battery
17. Display the simulation in 2D and 3D views, with live status for the selected drone and a time-stamped event log
18. Let the user start, pause, reset, and change the speed of the simulation
19. Produce identical results when the same scenario is run again with the same settings

Non-Functional Requirements

- **Performance:** The physics engine runs at 100 steps per simulated second, and the interface stays at 60 frames per second or more with at least 20 drones on the reference machine (Apple M4 Max), and at 30 frames per second or more on every team machine
- **Accuracy:** Physics results match known formulas: free fall within 0.1 m after 2 seconds, hovering drift below 0.000001 m over 10 seconds, and terminal velocity within 0.1%
- **Portability:** Builds and runs from a fresh clone on macOS on Apple Silicon and Windows on x64, verified by CI on both platforms
- **Determinism:** The same inputs always give the same outputs, on the same machine and across macOS and Windows; all randomness comes from a seeded generator
- **Maintainability:** Clean MVC separation; the model contains no user interface code; new drone types, forces, controllers, and event listeners can be added without changing existing classes
- **Robustness:** An error in one event listener does not stop the simulation or affect other listeners
- **Testability:** At least 80% line coverage on the model, with every model class unit tested
- **Documentation:** Javadoc on every public class and method, a README with run instructions, and up-to-date UML diagrams
- **Usability:** A first-time user can add a drone, build a three-step mission, and run it within 5 minutes without instructions
- **Reliability:** Invalid input (such as negative mass or throttle above 100%) is rejected with clear error messages instead of crashing the application

Estimated Timeline

1. Planning and Setup (1 week)

Week 1:

- Set up the Maven project, GitHub repository, branch protection, and CI on macOS and Windows
- Finalize requirements and SRS documentation
- Agree on core interfaces and create the first UML class diagram
- Verify every team member can run a basic JavaFX window

2. Development: Core Simulation (2 weeks)

Week 2:

- Implement Vector3D, gravity, drag, and the physics engine with semi-implicit Euler integration
- Implement the drone, quadcopter, and battery classes
- Create a console view and physics tests (free fall, terminal velocity)

Week 3:

- Implement ground contact, landing, and crash detection
- Connect battery drain to throttle
- Build the fixed-time-step simulation loop and the first JavaFX side view
- Milestone: version 1 core complete

3. Development: Control, Missions, and Events (3 weeks)

Week 4:

- Implement flight controllers (hover, altitude hold, waypoint)
- Support multiple drones, and add the top-down view and status panel

Week 5:

- Implement missions using the Command pattern and the mission queue
- Save and load missions from files
- Build the mission editor panel
- Replace the simple status values with the State pattern
- Add simulation controls (add drone, start mission, pause, speed)

Week 6:

- Implement the event system (Observer pattern) and the event log
- Add the return-home policy on low battery
- Add wind with gusts, air density, obstacles, no-fly zones, and collision detection
- Mid-project review of code, coverage, and diagrams
- Milestone: feature-complete simulation

4. Development: Extensions and 3D View (2 weeks)

Week 7:

- Add hexacopter and fixed-wing drones (lift and stall), with a flight controller for fixed-wing flight
- Add mission validation against drone capabilities
- Add a builder for drone creation
- Add payloads and the drop-payload mission step
- Bug fixing and Windows verification

Week 8:

- Build the JavaFX 3D view with camera and lighting
- Allow switching between 2D and 3D views without model changes
- Create the scripted demo scenario
- Milestone: 3D view complete

5. Stretch Goals and Polish (1 week)

Week 9:

- If all milestones are complete, add one stretch goal (RK4 integration, tilt-based flight, ode4j adapter, or jMonkeyEngine prototype)
- Otherwise, finish remaining work
- Complete Javadoc, raise test coverage, and draft the design document

6. Testing, Documentation, and Delivery (1 week)

Week 10:

- Code freeze: bug fixes only
- Run end-to-end scenario tests covering every functional requirement
- Acceptance test: someone outside the team runs the demo scenario using only the README
- Final design document, README, and UML diagrams
- Coverage report and fresh-clone test on all team machines
- Create demo video and presentation
- Project final submission